

XRPL Validator — Daily Engineering Report • 2026-08-21

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **14 commits** (xrpl-validator 13, xrpl-sdk 1)

Engineering changes

- **feat(xrpl-ledger): PreviousTxnID/PreviousTxnLgrSeq stamping + post-state threading census** (9876271 , xrpl-validator) Byte-for-byte phase 1: the engine now THREADS. New ledger::threading — stamp_threading marks every materially-changed write with the touching tx's hash and the ledger seq, under rippled's ApplyStateTable rule: a content-equal write-back (modulo pointer spelling and existing threading) is dropped from the meta and never threaded, so it is never stamped. 28 of 30 entry types carry the fields (fixPreviousTxnID added ...
- **feat(xrpl-ledger): threadOwners — created/deleted nodes thread their owner roots** (48367bc , xrpl-validator) Both census finds, one rule (ApplyStateTable.cpp:640-668): every CREATED or DELETED node threads the transaction to its owner accounts' AccountRoots — both limit issuers for a RippleState, else sfAccount and sfDestination when present, nothing for an AccountRoot. Those roots become the meta's pure-threading ModifiedNodes: the FinalFields==pre refreshes the expected-side filter has always dropped.
- **feat(xrpl-ledger): MPT issuance lifecycle — Create/Destroy/Set/Authorize, stubs retired** (64ebb07 , xrpl-validator) Byte-for-byte phase 2, first tranche: the four MPT lifecycle transactors leave STUB_TYPES and are judged like everything else.
- **feat(xrpl-ledger): DID, PermissionedDomain, AMMClawback — five more stubs retired** (75b2517 , xrpl-validator) Zero mainnet specimens exist for any of these in every fixture window, so these are BLIND SOURCE PORTS — flagged as such in-code; the scout becomes the verifier the day one lands. A faithful port can silently match; a fee-only stub is GUARANTEED to diverge.
- **feat(xrpl-ledger): XChain — all eight XLS-38 transactors; STUB_TYPES is EMPTY** (6036cee , xrpl-validator) The last eight stubs. ZERO mainnet usage exists (no Bridge object on mainnet, no XChain tx in any fixture window), so the only reachable branch today is 'no bridge' (tecNO_ENTRY) — everything deeper executes only after a XChainCreateBridge lands, which the scout would surface with a fixture. Blind source ports from XChainBridge.cpp (2390 lines, all eight transactors in one file): - keylets: bridge = ...
- **feat(xrpl-node): BYTE CENSUS — canonical-codec serialization vs ledger_entry binary=true** (a0b42aa , xrpl-validator) Byte-for-byte phase 3, the instrument: DX_BYTECHECK=1 serializes every surviving touched node through the SDK's canonical STObject codec (xrpl_core::codec::encode_transaction_json — its definitions table covers ledger fields, LedgerEntryType string resolution, Vector256, inner arrays/objects) and byte-compares against the real post-state blob from ledger_entry binary=true. Reports first-diff offset with hex context, ...

- **feat(xrpl-ledger): byte backlog to ZERO — 98 → 0, ten fixtures byte-exact** (1733d67 , xrpl-validator) Three grind rounds against the byte census, every fix a canonical-shape rule the meta-level gates could never see: ROUND 1 (98 → 40): directory zero-link pollution — relinks wrote explicit 'IndexNext: 0' onto hydrated pages and every fresh page carried both zero links; fresh pages now omit them (set_link). Fresh BOOK pages gain their canonical fields (ExchangeRate = the root key's low 8 bytes + TakerPays/Gets ...
- **feat(xrpl-ledger): pseudo-transactions + flag-ledger NegativeUNL rotation — phase 4 opens** (1083ba5 , xrpl-validator) UNLModify / SetFee / EnableAmendment (Change.cpp), the consensus-injected transactions with no Account, no Fee, no Sequence. dispatch::is_pseudo routes them past preclaim/apply_common entirely; build_txfields accepts the zero account. New keylets: amendments (dedup'd — one already existed), negativeUNL. The NegativeUNL/Amendments singletons hydrate on every fixture alongside FeeSettings.
- **fix(xrpl-ledger): self-cross deletions roll back with a failed strand — dead reaps stay banked** (9475a92 , xrpl-validator) Scan 4's single find, l106453801: a mirrored mis-attribution. 2D468DC2's REJECTED bridge candidate had walked leg A and self-cross-deleted the taker's own resting offer A20C602E (plus its book page); our rollback re-applied every reap from the failed candidate, so the deletion landed in 2D468DC2's meta (8v6 extra) — while mainnet still holds the offer until EA6D14A1 places on that very book ten txs later and ...
- **feat(xrpl-node): state_verify — snapshot-dir account_hash verification** (05de843 , xrpl-validator) Phase 4 substrate check: reads snapshot.py's harvested batches ({index, data} hex pairs), builds the State SHAMap with the canonical leaf formula (SHA512Half(MLN0 || data || key) — verify_rocks' and the live validator's), and compares the computed root against the pinned ledger's account_hash. Proves snapshot + SHAMap + hashing end-to-end before anything replays on top.
- **fix(core): full-state codec parity — STAmount getText spelling, MPT amounts, BaseTen u64s, 62-bit XRP range** (e8a2acb , xrpl-sdk) Fixes surfaced by replaying mainnet ledger #106455035 over the full 19,376,944-object state snapshot (decode → canon → encode must reproduce every blob byte-for-byte; census now 0 mismatches, 0 decode errors): - IouAmount::to_decimal mirrors STAmount::getText exactly: exponents outside [-25, -5] (and != 0) print as 'e' instead of a plain expansion the parser could not re-read (32,442 objects). - ...
- **feat(node): state_replay — full-ledger native replay vs mainnet account_hash** (83858f9 , xrpl-validator) Phase 4 capstone. Loads the entire 19,376,944-object verified snapshot, decodes every blob through the canonical codec (the load doubles as a full-state codec census: decode → canon → encode must reproduce every blob — 0 mismatches after the xrpl-core amount fixes), hexifies to the engine's account convention, applies the next ledger's tx set through the native engine with NO truth-overlay and NO hydration, stamps ...
- **fix(xrpl-ledger): six mainnet-parity fixes from full-ledger replay rounds 2-3** (e84d0b3 , xrpl-validator) Replaying #106455035-37 against account_hash (state_replay) surfaced six engine gaps the fixture corpus never reached; with all six the streak is three consecutive ledgers byte-for-byte (518 txs): - payment: a LEADING issuer-only path element (type 32) seeds from the spend currency — toStrand's normalization walks src → SendMax issuer first and the pairwise emission builds the same-currency cross-issuer book. ...
- **fix(xrpl-ledger): tfSell residual rests the subtracted in-side, repriced out** (26b2e2d , xrpl-validator) OfferCreate.cpp:491-521: after crossing, a tfSell offer's remainder keeps afterCross.in = TakerGets minus the crossed NET (a 16-digit STAmount subtraction), and REPRICES the out side — divRoundStrict(afterCross.in, rate, out.asset, roundUp=false), whole-drop floor on an XRP leg. Our placement derived the GETS side from rem_pays for both modes (right for buys — E7399DA3 — backwards for sells).

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Oracle-Triggered Orders via Sorted Directories** (dex) Add a tfTrigger flag to OfferCreate plus two SFields, sfTriggerPrice and sfOracleID (a keylet reference into an existing PriceOracle), creating a Stop
- **Sponsorship Transfer for Client Offboarding** (custody) Add a SponsorTransfer transaction (ttSPONSOR_TRANSFER) carrying sfObjectID and sfNewSponsor. Preclaim requires the incoming sponsor to hold a standing
- **Oracle-Ordered Liquidation Fencing for Vaults** (lending) Add a monotonic sfUpdateCounter SField to PriceOracle, incremented exactly once per ledger in which it is updated, with the increment order fixed by f

Appendix — commits

Time	Repo	Commit	Subject
07:42	xrpl-validator	9876271	feat(xrpl-ledger): PreviousTxnID/PreviousTxnLgrSeq stamping + post-state threading census
08:20	xrpl-validator	48367bc	feat(xrpl-ledger): threadOwners — created/deleted nodes thread their owner roots
08:45	xrpl-validator	64ebb07	feat(xrpl-ledger): MPT issuance lifecycle — Create/Destroy/Set/Authorize, stubs retired
12:05	xrpl-validator	75b2517	feat(xrpl-ledger): DID, PermissionedDomain, AMMClawback — five more stubs retired
12:16	xrpl-validator	6036cee	feat(xrpl-ledger): XChain — all eight XLS-38 transactors; STUB_TYPES is EMPTY
12:46	xrpl-validator	a0b42aa	feat(xrpl-node): BYTE CENSUS — canonical-codec serialization vs ledger_entry binary=true
17:28	xrpl-validator	1733d67	feat(xrpl-ledger): byte backlog to ZERO — 98 → 0, ten fixtures byte-exact
18:41	xrpl-validator	1083ba5	feat(xrpl-ledger): pseudo-transactions + flag-ledger NegativeUNL rotation — phase 4 opens
18:51	xrpl-validator	9475a92	fix(xrpl-ledger): self-cross deletions roll back with a failed strand — dead reaps stay banked
19:26	xrpl-validator	05de843	feat(xrpl-node): state_verify — snapshot-dir account_hash verification
22:02	xrpl-sdk	e8a2acb	fix(core): full-state codec parity — STAmount getText spelling, MPT amounts, BaseTen u64s, 62-bit XRP range
22:02	xrpl-validator	83858f9	feat(node): state_replay — full-ledger native replay vs mainnet account_hash
22:56	xrpl-validator	e84d0b3	fix(xrpl-ledger): six mainnet-parity fixes from full-ledger replay rounds 2-3
23:21	xrpl-validator	26b2e2d	fix(xrpl-ledger): tfSell residual rests the subtracted in-side, repriced out